SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING **NAME : E.LOHITHA**

COURSE CODE: CSA4303 **REG NO:192411103**

MARKS:90

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
 <input type="text" name="name">
 <input type="email" name="email">
 <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure. 2. Modify the structure to make it **standards-compliant**.

3. Interpret the **HTTP request generated** during form submission.

4. Analyze the **HTTP response cycle** from server to browser.

5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
 <head>
```

```
<title>Company</title>
</head>
<body>
<div>
<h1>Welcome
<p>About us
</div>
</body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
<input type="text" name="username">
<input type="password" name="password">
<button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I _E.LOHITHA/192411103_ *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand*

that any violation of academic integrity rules will result in disciplinary action. ***Signature of the student:*****E.LOHITHA**

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**	**MARKS**
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions	22.5
Explanation	25%	Detailed, well explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation	22.5
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis	18
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers	13.5
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps	9

Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand	4.5

ANSWERS

1 Online Symposium Portal

Questions:

1.1 Identify appropriate HTML5 semantic elements missing in the structure.

The given HTML form contains only a <form> element and input fields. It does not provide a complete semantic page structure. The following HTML5 elements can improve the structure and accessibility of the portal.

1. **<header>:** Defines the introductory content or heading of the webpage.
2. **<nav>:** Defines a section containing navigation links.
3. **<main>:** Represents the main content of the webpage.
4. **<section>:** Groups related content, such as registration details.
5. **<footer>:** Contains copyright information and contact details.
6. **<label>:** Provides descriptive labels for input fields.
7. **<fieldset>:** Groups related form controls.
8. **<legend>:** Provides a caption for a fieldset.

These elements improve readability, accessibility, maintainability, and compliance with HTML5 standards.

1.2 Modify the structure to make it standards-compliant.

The following code uses HTML5 semantic elements, proper form labels, validation attributes, and a correct document structure.

<!DOCTYPE html>

<html lang="en">

<head>

```
    <meta charset="UTF-8">     <meta name="viewport"
content="width=device-width, initial-scale=1.0">
    <title>Symposium Registration</title>
</head>
<body>
    <header>
        <h1>College Online Symposium</h1>
        <p>Register to participate in our symposium.</p>
    </header>
    <main>
        <section>
            <h2>Student Registration</h2>
            <form action="/register" method="post">
                <fieldset>
                    <legend>Registration Details</legend>
                    <label for="name">Full Name:</label>
<input type="text" id="name"
name="name" required>
                    <br><br>
                    <label for="email">Email Address:</label>
<input type="email" id="email"
name="email" required>                    <br><br>
                    <button type="submit">
                        Submit Registration
                    </button>
                </fieldset>
            </form>
        </section>
    </main>
```

```
    <footer>
        <p>Copyright 2026 College Symposium</p>
    </footer>
</body>
</html>
```

Explanation:

- <!DOCTYPE html> declares the HTML5 document type.
- <html lang="en"> specifies the document language.
- <meta charset="UTF-8"> defines the character encoding.
- <meta name="viewport"> supports responsive display on mobile devices.
- <form> defines the registration form.
- action="/register" specifies the server endpoint.
- method="post" sends form data in the HTTP request body.
- required ensures that the fields are completed before normal browser submission.
- <label> improves accessibility by identifying each input field.

1.3 Interpret the HTTP request generated during form submission.

When a student enters their name and email address and clicks the Submit button, the browser validates the form and generates an HTTP POST request.

The request is sent to the /register endpoint on the web server.

Example HTTP request:

```
POST /register HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded
name=John+Doe&email=john%40example.com
```

Explanation of the request components:

1. **POST:** Specifies the HTTP method used to submit the form data.
2. **Request URL:** /register identifies the server endpoint.
3. **Host:** Identifies the target web server.
4. **Content-Type:** Specifies the format of the submitted data.

5. **Request body:** Contains the student's name and email address.

The browser encodes the form data and sends it to the server. The server then processes the submitted information.

Unlike GET, POST normally places the submitted form data in the request body rather than in the URL. However, POST alone does not encrypt the data; HTTPS is required for encryption during transmission.

1.4 Analyze the HTTP response cycle from server to browser.

The HTTP response cycle consists of the following steps:

1. **User input:** The student enters registration details.
2. **Form validation:** The browser checks required fields and email format.
3. **HTTP request:** The browser sends a POST request to /register.
4. **Server processing:** The server receives and validates the submitted data.
5. **Data storage:** The server may store the registration information in a database.
6. **HTTP response:** The server returns a response containing a status code and, when appropriate, response content.
7. **Browser rendering:** The browser displays the response to the student.

Example HTTP response:

HTTP/1.1 200 OK

Content-Type: text/html

<!DOCTYPE html>

<html lang="en">

<body>

<h1>Registration Successful</h1>

Your registration has been received.

</body>

</html>

The status code 200 OK indicates that the request was successfully processed. Another common approach is to return a 303 See Other response that redirects the browser to a registration confirmation page.

Thus, HTTP enables communication between the browser and server through requests and responses.

1.5 Suggest improvements for usability and accessibility.

The following improvements can make the symposium portal easier to use and more accessible.

1. **Use labels:** Every input field should have a descriptive label.
2. **Apply validation:** Use required, type="email", and other suitable HTML validation attributes.
3. **Provide error messages:** Display clear messages when the submitted data is invalid.
4. **Use meaningful buttons:** Use text such as Submit Registration instead of unclear labels.
5. **Support keyboard navigation:** All form controls should be accessible using the keyboard.
6. **Maintain color contrast:** Text and backgrounds should have sufficient contrast.
7. **Use responsive design:** The page should work properly on mobile phones, tablets, and desktops.
8. **Display confirmation:** Show a success message after registration.
9. **Protect user information:** Use HTTPS and validate data on the server.
10. **Use semantic HTML:** Semantic elements improve document organization and assistive technology support.

These improvements provide a user-friendly, accessible, and secure registration system.

2. CROSS-BROWSER COMPATIBILITY ISSUE

2.1 Identify improper nesting and missing closing tags.

The given HTML code contains the following problems:

```
<html>
<head>
  <title>Company</title>
</head>
<body>
  <div>
```

```
    <h1>Welcome
    <p>About us
  </div>
</body>
</html>
```

Errors identified:

1. The <!DOCTYPE html> declaration is missing.
2. The <html> element does not specify the document language.
3. The closing tag </h1> is missing.
4. The closing tag </p> is missing.
5. The viewport metadata is missing for responsive display.
6. The page uses a generic without providing semantic meaning.

A corrected basic structure is:

```
<div>
  <h1>Welcome</h1>
  <p>About us</p>
</div>
```

Properly closing and nesting elements improves document validity and makes the HTML easier to maintain.

2.2 Analyze how different browsers may interpret this structure.

Different browsers use HTML parsing algorithms to interpret webpage content. When HTML contains missing tags or invalid markup, browsers may attempt to repair the document automatically.

However, relying on automatic correction may cause unexpected results.

The following problems may occur:

1. **Different DOM structures:** Browser parsing and error recovery can affect the resulting document structure.
2. **Layout differences:** Invalid markup may interact unexpectedly with CSS.
3. **CSS problems:** Selectors may match elements differently than expected.

4. **JavaScript errors:** Scripts that depend on a particular DOM structure may behave incorrectly.
5. **Accessibility issues:** Assistive technologies may not interpret the content as intended.
6. **Maintenance difficulties:** Developers may find it harder to debug invalid code.

Although modern browsers generally handle common HTML errors consistently, developers should always write valid HTML to reduce compatibility problems.

2.3 Identify missing semantic elements.

The following semantic elements can improve the company website.

1. **<header>:** Contains the website heading or introductory content.
2. **<nav>:** Contains links to Home, About Us, and Contact pages.
3. **<main>:** Contains the primary content of the webpage.
4. **<section>:** Groups related information, such as company details.
5. **<article>:** Represents independent content, such as a company news article.
6. **<footer>:** Contains copyright information and contact details.

For example, a company introduction can be placed inside <main> and <section> instead of using only generic elements.

Semantic HTML improves code organization, accessibility, and maintainability.

2.4 Provide a corrected W3C-compliant HTML structure.

The following HTML document follows standard HTML5 document structure and uses semantic elements appropriately.

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">    <meta name="viewport"

content="width=device-width, initial-scale=1.0">

  <title>Company Website</title>

</head>

<body>

  <header>

    <h1>Welcome to Our Company</h1>
```

```
        <nav aria-label="Main navigation">
            <a href="#home">Home</a>
            <a href="#about">About Us</a>
            <a href="#contact">Contact</a>
        </nav>
    </header>
    <main id="home">
        <section id="about">
            <h2>About Us</h2>
            <p>
                Our company provides quality products
and services to our customers.
            </p>
        </section>
        <section id="contact">
            <h2>Contact Us</h2>
            <p>Email: info@example.com</p>
        </section>
    </main>
    <footer>
        <p>Copyright 2026 Our Company. All rights reserved.</p>
    </footer>
</body>
</html>
```

Explanation:

- <!DOCTYPE html> enables standards-based HTML parsing.
- <html lang="en"> identifies the page language.
- <meta charset="UTF-8"> specifies character encoding.
- The viewport metadata supports mobile-friendly display.

- <header>, <nav>, <main>, <section>, and <footer> provide meaningful page structure.
- All elements are properly nested and closed.

The document can also be checked using the W3C Markup Validation Service to identify HTML errors.

2.5 Suggest techniques to ensure cross-browser compatibility.

The following techniques help websites work consistently across browsers.

1. **Follow web standards:** Use valid HTML5 and CSS.
2. **Declare the document type:** Include <!DOCTYPE html> to enable standards mode.
3. **Close all elements:** Use proper nesting and valid attributes.
4. **Use CSS normalization:** Use a reset or normalization stylesheet when appropriate to reduce default-style differences.
5. **Test multiple browsers:** Test the website in Chrome, Firefox, Safari, and Edge.
6. **Use responsive design:** Apply CSS media queries for different screen sizes.
7. **Check feature support:** Use browser compatibility references such as MDN Web Docs.
8. **Use feature detection:** Check whether a browser supports a feature before relying on it.
9. **Validate HTML and CSS:** Use validation tools to identify errors.
10. **Test JavaScript:** Avoid relying on browser-specific behavior.
11. **Use progressive enhancement:** Ensure essential content and functions work even if advanced features are unavailable.
12. **Test accessibility:** Check keyboard navigation, labels, and screen-reader support.

By applying these techniques, developers can create websites that are reliable, accessible, and consistent across different browsers.

3. FORM SUBMISSION FAILURE

3.1 Identify issues in the form configuration.

The given HTML code is:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
```

<button type="button">Login</button>

</form>

The following issues are present:

1. The button uses type="button" instead of type="submit".
2. Clicking the button does not automatically submit the form.
3. The GET method is inappropriate for submitting login credentials because the data appears in the URL.
4. The input fields do not have associated labels.
5. The fields do not have required attributes.
6. The form does not specify a secure transmission method by itself.

The main reason for the submission failure is the incorrect button type.

3.2 Analyze why the HTTP request is not triggered.

The form contains:

<button type="button">Login</button>

A button with type="button" performs no automatic form-submission action. It can be used for custom JavaScript functionality, but it does not submit the form by itself.

Therefore, clicking Login does not automatically send a form submission request to /submit.

The problem can be corrected by using:

<button type="submit">Login</button>

When the user clicks the Submit button, the browser validates the form and submits it unless validation fails or JavaScript or other browser behavior prevents submission.

The form then generates an HTTP request according to its action and method attributes.

3.3 Interpret the role of GET method in this scenario.

The GET method is used mainly to retrieve resources from a server. When a form uses GET, its successful form controls are encoded as query parameters in the URL.

For example:

/submit?username=student&password=1234

A corresponding request may look like:

GET /submit?username=student&password=1234 HTTP/1.1

Host: example.com

Disadvantages of using GET for login:

1. Form data appears in the URL.
2. URLs may be stored in browser history.
3. URLs may be recorded in server logs.
4. URLs may be exposed through analytics or other URL-handling mechanisms.
5. Sensitive credentials may be exposed unintentionally.

Therefore, GET should not be used to submit passwords.

The POST method is more appropriate for login forms because it sends the form data in the request body. However, POST does not encrypt the data; HTTPS is necessary to protect it during transmission.

3.4 Propose a corrected implementation.

The following code uses POST, proper labels, required fields, and a submit button.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">    <meta name="viewport"
content="width=device-width, initial-scale=1.0">
  <title>Student Login</title>
</head>
<body>
  <main>
    <section>
      <h1>Student Login</h1>
      <form action="/submit" method="post">
        <div>
          <label for="username">Username:</label>
          <input type="text"
id="username"
name="username"
```

```
autocomplete="username"
required>
            </div>

            <br>
            <div>
                <label for="password">Password:</label>
                <input type="password"
id="password"                    name="password"
autocomplete="current-password"
required>
            </div>
            <br>
            <button type="submit">Login</button>
        </form>
    </section>
  </main>
</body>
</html>
```

Explanation:

- action="/submit" specifies the server endpoint.
- method="post" sends form data in the request body.
- type="password" masks the password characters on the screen.
- type="submit" triggers form submission.
- required ensures that both fields are completed before normal browser submission.
- <label> elements improve accessibility.
- autocomplete helps browsers and password managers identify the fields.

The server must still verify the credentials and return an appropriate response, such as a successful login or an authentication error.

3.5 Suggest improvements for secure data transmission.

The following security measures should be implemented to protect login information.

1. **Use HTTPS:** HTTPS encrypts communication between the browser and server, protecting credentials from interception during transmission.

2. **Use POST:** Submit usernames and passwords in the request body instead of placing them in the URL.

3. **Validate data on the server:** Client-side validation improves usability, but the server must independently validate all submitted data.

4. **Hash passwords securely:** Store passwords using an established password-hashing algorithm such as Argon2id, bcrypt, or scrypt. Never store passwords as plain text.

5. **Use secure session cookies:** Set suitable cookie attributes such as Secure, HttpOnly, and SameSite.

6. **Prevent SQL injection:** Use parameterized queries or prepared statements when interacting with databases.

7. **Protect against CSRF:** Apply appropriate CSRF protection where required by the application's authentication and cookie design.

8. **Prevent brute-force attacks:** Use rate limiting, login-attempt monitoring, and other suitable protections.

9. **Avoid exposing credentials:** Do not include passwords in URLs, logs, error messages, or unnecessary client-side scripts.

10. **Manage sessions safely:** Regenerate session identifiers after successful authentication and expire sessions appropriately.

These measures improve confidentiality, authentication security, and protection against common web attacks.